

操作系统设计任务书

Linux 桌面 Agentic OS 设计与实现

一、课程基本信息

课程名称	操作系统设计
课程性质	专业必修课（实践环节）
学分/学时	2 学分 / 4 周（64 学时）
授课对象	计算机科学与技术专业（大三进大四年级）
学期	夏季学期
指导教师	陈浩，肖德贵

二、设计目的

1. 核心目标

在 Linux 桌面环境中实现类似微软 UF0² 的桌面 Agentic OS 原型，实现自然语言驱动的桌面操作自动化，并在实践中理解操作系统的三大核心原理：虚拟化、并发、持久性。

2. 课程目标

通过 AI 智能体与操作系统原理的深度融合，培养学生以下能力：

（1）理解操作系统核心原理：虚拟化（进程调度、资源隔离）、并发（异步执行、同步机制）、持久性（文件系统、存储管理）

（2）掌握 AI 智能体架构设计：学习微软 UF0² 等前沿桌面 Agent 架构，理解 HostAgent、AppAgent、Knowledge Substrate 等组件设计

（3）提升系统工程能力：通过模块化设计、接口契约、协作开发，锻炼大型软件系统开发能力

（4）培养团队协作精神：通过分组协作、进度对齐、接口联调，体验真实软件开发流程

（5）探索 OS 新形态：思考操作系统如何为 AI 智能体提供原生支持，理解 Agentic OS 概念

三、设计任务

本项目基于微软 UFO² (UI-Focused Agent) 架构，在 Linux 桌面环境实现一个支持自然语言交互的 Agentic OS 原型系统。

1. UFO² 架构简介

微软 UFO² 是专为 Windows 桌面设计的 AI 智能体框架，核心组件包括：

- (1) HostAgent (主机智能体)：接收用户自然语言指令，意图理解与任务分发，协调多个 AppAgent 协作
- (2) AppAgent (应用智能体)：专注单一应用的操作执行，状态管理与动作规划，Hybrid GUI + API Actions
- (3) Control Detection (控件检测)：Visual Detection 视觉检测，UIA Detection 结构化检测，混合检测提高鲁棒性
- (4) Knowledge Substrate (知识底座)：执行轨迹存储与检索，应用文档知识库，支持 RAG 检索增强

2. 本项目任务

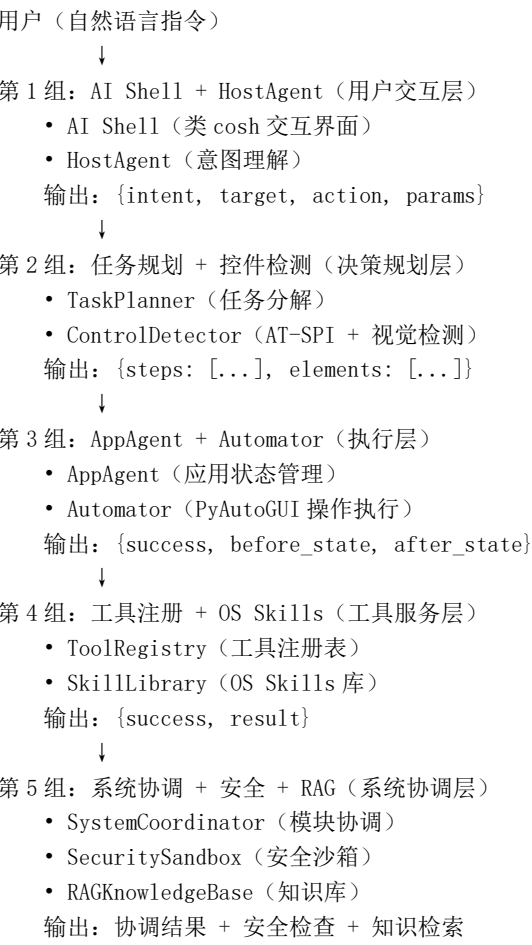
- (1) 调研 UFO² 架构原理，理解 HostAgent、AppAgent、Control Detection、Knowledge Substrate 等核心组件
- (2) 设计 Linux 桌面 Agentic OS 架构，将 Windows 的 UIA 适配到 Linux 的 AT-SPI/D-Bus
- (3) 实现 AI Shell (类似 Agentic OS 的 cosh)，作为用户交互入口
- (4) 实现任务规划模块，将自然语言指令分解为 GUI 操作序列
- (5) 实现控件检测模块，使用 AT-SPI 和视觉检测混合方式定位 UI 元素
- (6) 实现 GUI 操作执行器，使用 PyAutoGUI 执行点击、输入、快捷键等操作
- (7) 实现工具注册与 OS Skills 库，封装文件操作等系统管理功能
- (8) 实现安全沙箱机制，防止危险操作执行
- (9) 实现 RAG 知识库，存储执行轨迹并支持检索

(10) 完成端到端演示，实现“整理下载目录”等复杂任务的自动化执行

四、分组安排

本项目共分 5 组，每组不超过 5 人。各组基于 UFO² 架构分工协作，最终集成完成完整系统。

1. 分组架构图



2. 分组详情表

组号	负责模块	对应 UFO² 组件	核心职责
第 1 组	AI Shell + HostAgent	HostAgent	用户交互 + 意图理解

第 2 组	任务规划 + 控件检测	Control Detection	任务分解 + UI 元素定位
第 3 组	AppAgent + Automator	AppAgent + Automator	应用状态管理 + GUI 操作执行
第 4 组	工具注册 + OS Skills	Tools	工具封装 + 技能定义
第 5 组	系统协调 + 安全 + RAG	Coordinator + Knowledge	模块协调 + 安全检查 + 知识检索

3. 组间协作关系

数据流向：
用户 → 第 1 组 → 第 5 组 (安全检查) → 第 2 组 → 第 3 组 → 第 4 组 ↔ 第 5 组

- 调用关系：
- 第 1 组 → 调用第 5 组安全检查 → 调用第 2 组任务规划
 - 第 2 组 → 调用第 3 组执行 → 调用第 4 组工具
 - 第 5 组 → 协调所有模块 → 审计记录 → RAG 存储

4. 接口契约表

组号	输出接口	格式示例
第 1 组	intent_json	{ "intent": "文件操作", "target": "...", "action": "...", "params": {...} }
第 2 组	plan_json	{ "steps": [{ "step_id": 1, "action": "...", "target": "..." }], "elements": [...] } }
第 3 组	result_json	{ "success": true, "before_state": "...", "after_state": "..." }
第 4 组	tool_result_json	{ "success": true, "result": "..." }
第 5 组	check_json	{ "approved": true, "risk_level": "low medium high critical" }

五、分组任务详解

1. 第 1 组：AI Shell + HostAgent

对应 UFO² 组件	HostAgent
所属层次	用户交互层
核心职责	自然语言交互 + 意图理解
OS 原理关联	虚拟化：HostAgent 调度多个 AppAgent，类似进程调度器
技术栈	Python + OpenAI API
代码量估计	150-200 行

(1) 功能要求

- AI Shell：实现类 `cosh` 的交互界面，支持自然语言输入和历史记录
- HostAgent：调用 LLM 进行意图理解，输出结构化任务描述
- 支持意图类型识别：文件操作、应用控制、系统设置、信息查询
- 支持参数提取：目标对象、动作、路径等参数解析
- 提供 Mock 实现，便于其他组独立开发

(2) 分周进度

周次	任务内容
第 1 周	调研 UFO² HostAgent 设计，定义接口格式，实现 Mock
第 2 周	实现 AI Shell 交互界面，实现意图理解模块
第 3 周	与第 5 组联调安全检查，与第 2 组联调任务分发
第 4 周	完善交互体验，参与整体系统集成

(3) 代码示例

```
# 第 1 组：AI Shell + HostAgent
import openai
import json

class AIShell:
    """AI Shell - 类似 Agentic OS 的 cosh"""
    def __init__(self, host_agent):
        self.host_agent = host_agent
```

```

self.history = []

def run(self):
    print("Linux Agentic OS Shell (cosh)")
    print("输入自然语言指令, 'exit' 退出")
    while True:
        user_input = input("\n> ")
        if user_input.lower() == 'exit':
            break
        task = self.host_agent.understand_intent(user_input)
        self.history.append({"user": user_input, "task": task})
        print(f"[Intent] {task['intent']} | Target: {task['target']}")

class HostAgent:
    """HostAgent - 意图理解"""
    def __init__(self, model="qwen-plus"):
        self.client = openai.OpenAI(
            base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
        )
        self.model = model

    def understand_intent(self, user_input: str) -> dict:
        prompt = f"分析指令提取意图, 输出 JSON: {user_input}"
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[{"role": "user", "content": prompt}]
        )
        return json.loads(response.choices[0].message.content)

if __name__ == "__main__":
    shell = AIShell(HostAgent())
    shell.run()

```

2. 第2组：任务规划 + 控件检测

对应 UFO ² 组件	Control Detection + 任务规划
所属层次	决策规划层
核心职责	任务分解 + UI 元素定位
OS 原理关联	并发：任务规划需处理异步执行、状态同步，类似并发调度
技术栈	Python + pyatspi + PIL + OpenAI API
代码量估计	200-250 行

(1) 功能要求

- TaskPlanner：将任务分解为 GUI 操作步骤序列

- ControlDetector：使用 AT-SPI 遍历控件树，获取元素位置
- 混合检测：AT-SPI 结构化检测 + 视觉检测（可选）
- 元素定位：支持按角色、名称查找 UI 控件
- 输出标准格式：steps 数组 + elements 字典

(2) 分周进度

周次	任务内容
第 1 周	调研 UF02 Control Detection，定义接口，实现 Mock
第 2 周	实现任务分解算法，实现 AT-SPI 控件检测
第 3 周	集成视觉检测，与第 3 组联调元素定位
第 4 周	完善混合检测，参与整体系统集成

(3) 代码示例

```
# 第 2 组：任务规划 + 控件检测
import pyatspi
from PIL import ImageGrab
import json

class TaskPlanner:
    """任务规划器"""
    def plan(self, task: dict) -> dict:
        if task['intent'] == '文件操作':
            return {"steps": [
                {"step_id": 1, "action": "open_app", "target": "Files"},
                {"step_id": 2, "action": "navigate", "target": task['params'].get('path', '/')},
                {"step_id": 3, "action": "click", "target": "button_new_folder"},
            ]}
        return {"steps": []}

class ControlDetector:
    """控件检测器 - AT-SPI 检测"""
    def __init__(self):
        self.desktop = pyatspi.Registry.getDesktop(0)

    def detect_elements(self) -> list:
        elements = []
        for app in self.desktop:
            for window in app:
                for i in range(min(window.childCount, 50)):
                    child = window.getChildAtIndex(i)
```

```

        pos = child.getPosition(pyatspi.XY_SCREEN)
        size = child.getSize()
        elements.append({
            "role": child.getRoleName(),
            "name": child.name or "",
            "bbox": [pos[0], pos[1], size[0], size[1]]
        })
    return elements

def find_element(self, role: str = None, name: str = None) -> dict:
    for el in self.detect_elements():
        if role and el["role"] != role:
            continue
        if name and name not in el["name"]:
            continue
        return el
    return None

if __name__ == "__main__":
    planner = TaskPlanner()
    detector = ControlDetector()
    task = {"intent": "文件操作", "params": {"path": "/home"}}
    print(planner.plan(task))

```

3. 第3组: AppAgent + Automator

对应 UFO ² 组件	AppAgent + Automator
所属层次	执行层
核心职责	应用状态管理 + GUI 操作执行
OS 原理关联	虚拟化: 每个 AppAgent 管理独立应用状态, 类似进程状态机
技术栈	Python + PyAutoGUI + subprocess
代码量估计	150-200 行

(1) 功能要求

- Automator: 执行 GUI 操作 (点击、输入、快捷键、拖拽)
- AppAgent: 管理应用状态, 执行操作序列
- 状态验证: 执行前后截图对比
- 错误处理: 执行失败时反馈给第 2 组调整
- FileManagerAgent: 特化实现文件管理器操作

(2) 分周进度

周次	任务内容
第 1 周	调研 UF0 ² AppAgent，定义接口，实现 Mock
第 2 周	实现 GUI 操作执行器，实现文件管理器 Agent
第 3 周	与第 2 组联调控件定位，与第 4 组联调工具调用
第 4 周	完善错误处理，参与整体系统集成

(3) 代码示例

第 3 组: AppAgent + Automator

```
import pyautogui
import subprocess
import time
from PIL import ImageGrab
```

```
class Automator:
```

```
    """Automator - GUI 操作执行器"""
```

```
    def execute_step(self, step: dict, element_info: dict = None) -> dict:
```

```
        action = step.get("action")
```

```
        if action == "click":
```

```
            bbox = element_info.get("bbox", [0, 0, 100, 100])
```

```
            x = bbox[0] + bbox[2] // 2
```

```
            y = bbox[1] + bbox[3] // 2
```

```
            pyautogui.click(x, y)
```

```
            return {"success": True, "message": f"点击({x}, {y})"}
```

```
        elif action == "type":
```

```
            text = step.get("params", {}).get("text", "")
```

```
            pyautogui.typewrite(text)
```

```
            return {"success": True}
```

```
        elif action == "hotkey":
```

```
            keys = step.get("params", {}).get("keys", [])
```

```
            pyautogui.hotkey(*keys)
```

```
            return {"success": True}
```

```
        return {"success": False, "error": f"未知操作: {action}"}
```

```
class FileManagerAgent:
```

```
    """文件管理器 Agent"""
```

```
    def navigate_to(self, path: str):
```

```
        pyautogui.hotkey('ctrl', 'l')
```

```
        time.sleep(0.5)
```

```
        pyautogui.typewrite(path)
```

```
        pyautogui.hotkey('enter')
```

```
def create_folder(self, name: str):
    pyautogui.hotkey('ctrl', 'shift', 'n')
    pyautogui.typewrite(name)
    pyautogui.hotkey('enter')

if __name__ == "__main__":
    automator = Automator()
    file_agent = FileManagerAgent()
```

4. 第 4 组：工具注册 + OS Skills

对应 UFO² 组件	Tools
所属层次	工具服务层
核心职责	工具封装 + 技能定义
OS 原理关联	持久性：文件系统操作、目录管理，直接体现文件系统原理
技术栈	Python + shutil + subprocess
代码量估计	150-200 行

(1) 功能要求

- ToolRegistry：管理所有可用工具，提供统一调用接口
- SkillLibrary：封装系统管理操作（文件整理、系统检查等）
- 工具签名：验证工具来源和完整性（可选）
- 调用统计：记录工具调用次数和结果
- 参考 MCP 协议设计工具接口

(2) 分周进度

周次	任务内容
第 1 周	调研 MCP 协议，定义接口，实现 Mock
第 2 周	实现工具注册表，实现文件操作工具
第 3 周	实现 OS Skills，与第 3 组联调
第 4 周	完善工具签名，参与整体系统集成

(3) 代码示例

```
# 第 4 组：工具注册 + OS Skills
import shutil
```



```

        shutil.move(str(file), str(path / cat / file.name))
    return "整理完成"

if __name__ == "__main__":
    registry = ToolRegistry()
    skills = SkillLibrary()
```

5. 第 5 组：系统协调 + 安全 + RAG

对应 UFO² 组件	Coordinator + Security + Knowledge Substrate
所属层次	系统协调层
核心职责	模块协调 + 安全检查 + 知识检索
OS 原理关联	虚拟化+并发+持久性：安全沙箱（资源隔离）、审计日志（持久化）、RAG 知识库（存储管理）
技术栈	Python + OpenAI API + numpy
代码量估计	200-250 行

(1) 功能要求

- SystemCoordinator：注册所有模块，编排执行流程
- SecuritySandbox：四层安全架构（权限检查→沙箱执行→签名验证→隐私保护）
- RAGKnowledgeBase：存储执行轨迹，支持向量检索
- 审计日志：记录所有操作，支持事后审计
- 降级方案：当其他组落后时提供备用 Mock 实现

(2) 分周进度

周次	任务内容
第 1 周	调研 UFO² Knowledge Substrate，定义接口，实现 Mock
第 2 周	实现系统协调器，实现安全检查模块
第 3 周	实现 RAG 知识库，与其他组联调
第 4 周	协调所有模块集成，完成端到端演示

(3) 代码示例

```
# 第 5 组：系统协调 + 安全 + RAG
import openai
```

```

import subprocess
import numpy as np
from datetime import datetime

class SystemCoordinator:
    """系统协调器"""
    def __init__(self):
        self.modules = {}

    def register(self, group_id: int, module):
        self.modules[group_id] = module

    def orchestrate(self, user_input: str) -> str:
        # 1. 意图理解
        task = self.modules[1].understand_intent(user_input)
        # 2. 安全检查
        if not self.modules[5].check_permission(task['action'], task['target'])['approved']:
            return "安全检查未通过"
        # 3. 任务规划
        plan = self.modules[2].plan(task)
        elements = self.modules[2].detect_elements()
        # 4. 执行
        results = self.modules[3].execute_plan(plan['steps'], elements)
        # 5. 审计
        self.modules[5].audit({"time": datetime.now().isoformat(), "input": user_input})
        return f"执行完成: {len(results)}步"

class SecuritySandbox:
    """安全沙箱"""
    DANGEROUS = ['rm', 'format', 'shutdown', 'dd']
    PROTECTED = ['/etc', '/usr', '/root', '/boot']

    def check_permission(self, action: str, target: str) -> dict:
        if any(d in action for d in self.DANGEROUS):
            return {"approved": False, "reason": "危险操作"}
        if any(p in target for p in self.PROTECTED):
            return {"approved": False, "reason": "受保护路径"}
        return {"approved": True}

    def audit(self, record: dict):
        print(f"AUDIT: {record['time']} | {record['input']}")

class RAGKnowledgeBase:
    """RAG 知识库"""
    def __init__(self):
        self.documents = []

    def add_trace(self, input: str, actions: list, result: str):

```

```
self.documents.append(f"Input: {input}\nActions: {actions}\nResult: {result}")

def query(self, query: str, k: int = 3) -> list:
    return self.documents[:k]

if __name__ == "__main__":
    coordinator = SystemCoordinator()
    security = SecuritySandbox()
    rag = RAGKnowledgeBase()
```

六、评分标准

本项目采用分项评分，总分 100 分。

1. 评分表

评分项	分值	评分说明
调研报告	20 分	UFO² 架构调研、AT-SPI 调研、相关论文阅读（不少于 3000 字）
设计文档	30 分	架构分析、接口设计、模块设计、流程图、可行性分析、风险评估
原型实现	20 分	每组核心功能实现、模块独立运行、单元测试
系统集成	20 分	5 组模块成功集成、数据流正确、端到端演示、联调测试
汇报演示	5 分	PPT 质量、演示效果、答辩表现
代码质量	5 分	代码规范、注释完整、测试覆盖

2. 评分细则

- （1）调研报告（20 分）：内容完整性 10 分 + 分析深度 7 分 + 格式规范 3 分
- （2）设计文档（30 分）：架构分析 8 分 + 接口设计 10 分 + 模块设计 8 分 + 可行性分析 4 分
- （3）原型实现（20 分）：功能实现 14 分 + 测试覆盖 4 分 + 代码规范 2 分

- (4) 系统集成 (20 分)：模块集成 10 分 + 联调测试 5 分 + 端到端演示 5 分
- (5) 汇报演示 (5 分)：PPT 质量 2 分 + 演示效果 2 分 + 答辩表现 1 分
- (6) 代码质量 (5 分)：PEP8 规范 2 分 + 注释完整 2 分 + 测试覆盖 1 分

七、验收标准

1. 必做验收要点

- (1) 调研报告：不少于 3000 字，包含 UFO² 架构分析、AT-SPI 调研、参考文献
- (2) 设计文档：架构分析、接口契约、模块设计、流程图、可行性分析、风险评估
- (3) 原型实现：每组模块能独立运行、单元测试通过、Mock 接口可用
- (4) 系统集成：5 组模块成功集成，数据流正确（第 1→2→3→4 组），端到端演示通过
- (5) 安全机制：危险操作被拦截（rm、format、shutdown 等）
- (6) 汇报演示：PPT 不少于 15 页、演示时间不少于 10 分钟

2. 演示场景表

场景	用户指令	GUI 操作步骤	验收标准
场景 1：基础 GUI	"打开文件管理器"	Super 键→输入 "Files"→点击图标	文件管理器打开成功
场景 2：导航操作	"导航到 /home/user/Documents"	Ctrl+L→输入路径→Enter	显示目标目录内容
场景 3：按钮点击	"创建 project 文件夹"	点击新建按钮→输入名称→Enter	文件夹创建成功
场景 4：文本编辑	"打开 test.txt 并添加一行"	双击文件→编辑→Ctrl+S	文件内容更新成功
场景 5：窗口切换	"切换到浏览器"	Alt+Tab→选择窗口	浏览器窗口激活
场景 6：复杂任务	"整理下载目录"	扫描→分类→创建文件夹→移动文件	文件按类型分类完成

3. 系统端到端流程说明

完整执行流程（以“打开文件管理器”为例）：

步骤 1：用户输入（用户）

- 用户在 AI Shell 输入自然语言指令：“打开文件管理器”

步骤 2：意图理解（第 1 组：AI Shell + HostAgent）

- HostAgent 调用 LLM 解析意图
- 输出：{"intent": "应用控制", "target": "Files", "action": "open"}

步骤 3：安全检查（第 5 组：系统协调 + 安全）

- SecuritySandbox 检查操作权限
- 输出：{"approved": true, "risk_level": "low"}

步骤 4：任务规划（第 2 组：任务规划 + 控件检测）

- TaskPlanner 分解为 GUI 操作步骤
- ControlDetector 检测桌面图标位置
- 输出：{"steps": [{"action": "click", "target": "Files 图标"}], "elements": {"Files 图标": {"bbox": [x, y, w, h]}}

步骤 5：操作执行（第 3 组：AppAgent + Automator）

- AppAgent 管理应用状态
- Automator 执行点击操作：PyAutoGUI.click(x+w/2, y+h/2)
- 输出：{"success": true, "before_state": "...", "after_state": "..."}

步骤 6：工具调用（第 4 组：工具注册 + OS Skills）

- 如需文件操作，调用 ToolRegistry
- 输出：{"success": true, "result": "..."}

步骤 7：审计记录（第 5 组：系统协调 + RAG）

- SystemCoordinator 记录执行轨迹
- RAGKnowledgeBase 存储到知识库
- 输出：完成，返回结果给用户

八、开发环境

1. 硬件环境

- （1）CPU：Intel i5 及以上或同等性能处理器
- （2）内存：8GB 及以上
- （3）硬盘：50GB 可用空间
- （4）显示器：支持 1920×1080 分辨率

2. 软件环境

类别	软件/工具	版本要求
操作系统	Ubuntu Linux	22.04 LTS 及以上
桌面环境	GNOME	42 及以上
编程语言	Python	3.10 及以上
GUI 自动化	PyAutoGUI	0.9.54 及以上
控件检测	pyatspi	2.38 及以上
LLM API	阿里云通义千问	qwen-plus
向量检索	numpy	1.24 及以上

3. 开发工具

- (1) IDE: VSCode / PyCharm / 任意 Python IDE
- (2) 调试: Python debugger + 截屏对比工具
- (3) 测试: pytest / unittest
- (4) 文档: Markdown 编辑器

4. 环境配置步骤

```
# 1. 安装 Python 依赖
pip install pyautogui pyatspi pillow openai numpy

# 2. 配置 AT-SPI 访问权限
# 在系统设置中启用辅助功能 (Accessibility)

# 3. 配置 LLM API 密钥
export OPENAI_API_KEY="your-api-key"
export OPENAI_BASE_URL="https://dashscope.aliyuncs.com/compatible-mode/v1"

# 4. 安装 bubblewrap (安全沙箱)
sudo apt install bubblewrap

# 5. 测试环境
python -c "import pyautogui; print(pyautogui.position())"
```

九、参考资料

1. 核心论文

- (1) UF0: A UI-Focused Agent for Windows OS Interaction. Microsoft Research, 2024. <https://arxiv.org/abs/2402.07939>
- (2) UF0²: Enhanced UI-Focused Agent with Hybrid GUI+API Actions. Microsoft Research, 2025. <https://arxiv.org/abs/2511.11332>
- (3) Agent Operating Systems (AOS): Integrating Agentic Control Planes into Traditional Operating Systems. arXiv, 2024. <https://arxiv.org/abs/2606.01508>
- (4) SeeAct: Generalist Web Agents with Grounded Vision-Language Interaction. 2024.

2. 技术文档

- (1) AT-SPI Documentation: <https://accessibility.linuxfoundation.org/>
- (2) PyAutoGUI Documentation: <https://pyautogui.readthedocs.io/>
- (3) GNOME Accessibility Guide: <https://help.gnome.org/>
- (4) MCP Protocol Specification: <https://modelcontextprotocol.io/>
- (5) Python unittest.mock:
<https://docs.python.org/3/library/unittest.mock.html>
- (6) Mock 设计实例 (RealPython) : <https://realpython.com/python-mock-library/>

3. 参考教材

- (1) 操作系统导论 (Operating Systems: Three Easy Pieces). Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau.
- (2) 现代操作系统 (Modern Operating Systems). Andrew S. Tanenbaum.
- (3) 深入理解计算机系统 (Computer Systems: A Programmer's Perspective). Randal E. Bryant, David R. O'Hallaron.

4. 相关开源项目

- (1) Microsoft UFO: UI-Focused Agent for Windows OS.
<https://github.com/microsoft/UFO>
- (2) 阿里 ANOLISA: Agentic Nexus Operating Layer & Interface System Architecture. <https://github.com/alibaba/anolisa>
- (3) Open Interpreter: Natural Language Computer Control.
<https://github.com/OpenInterpreter/open-interpreter>

十、提交要求

1. 检查点时间表

检查点	时间	提交材料
检查点 1	第 1 周结束	接口文档 + Mock 代码 + 调研报告初稿
检查点 2	第 2 周结束	模块代码 + 设计文档 + 单元测试报告
检查点 3	第 3 周结束	联调测试报告 + 集成演示视频
检查点 4	第 4 周结束	完整系统 + PPT 汇报 + 最终报告

2. 提交格式要求

- (1) 报告格式: PDF 文档, 正文使用宋体小四号字
- (2) 代码格式: Python 源码 (.py), 带注释和 README
- (3) 演示视频: MP4 格式, 时长不超过 5 分钟
- (4) PPT 格式: PPTX 格式, 不少于 15 页
- (5) 打包方式: 每组提交一个 ZIP 压缩包, 命名为“组号_组长姓名.zip”

十一、注意事项

- (1) 组间协作: 各组必须严格遵守接口契约, 确保模块能正确集成

- (2) 进度对齐：每周检查点必须按时完成，落后组需要及时沟通
- (3) Mock 机制：每组必须提供 Mock 实现，便于其他组独立开发
- (4) 安全测试：危险操作必须在安全沙箱中测试，不得在真实系统上执行
- (5) 代码规范：代码必须带注释，遵循 Python 编码规范（PEP8）
- (6) 学术诚信：调研报告必须标注参考文献，不得抄袭
- (7) 备份机制：代码和文档必须定期备份，防止丢失